

Planificador de Estudios con Prerrequisitos

Proyecto Final – Estructura de Datos II

Ingeniería Informática, 2do año

Wandiliel Torres Hernández

2026

Descripción de la problemática

Bueno, la idea surge de algo que todos los estudiantes de informática vivimos: planificar las materias de la carrera sin saber bien cuál va primero o cuál necesita qué. Muchas veces nos inscribimos en una materia y después nos damos cuenta que necesitábamos haber visto otra antes, y ahí empiezan los problemas. Por eso me pareció útil hacer un sistema que ayude a organizar el plan de estudios.

El objetivo específico es crear una herramienta que permita guardar la jerarquía de materias (por ejemplo, por departamento o área) y también las dependencias entre ellas, o sea, los famosos prerrequisitos. Así un estudiante podría consultar en qué orden le conviene cursar, o ver qué necesita antes de meterse en una materia difícil.

¿Por qué usar árboles y grafos? El árbol sirve para la parte jerárquica, porque las materias se agrupan naturalmente: una facultad tiene varios departamentos, cada departamento tiene áreas, y dentro de esas áreas están las materias. Es como una estructura de padre e hijos, ideal para un árbol general.

El grafo, en cambio, es perfecto para los prerrequisitos, porque una materia puede depender de varias otras y esas a su vez de más, formando una red de dependencias. Además esas dependencias no pueden tener ciclos (una materia no puede ser requisito de sí misma), así que usamos un grafo dirigido acíclico (DAG). Con el ordenamiento topológico podemos generar una secuencia de estudio que respete todos los prerrequisitos.

En resumen, el árbol organiza la información por áreas y el grafo modela las reglas de cursado. Los dos se comunican mediante el ID de cada materia, que es el mismo en ambas estructuras. Así podemos buscar una materia en el árbol y luego preguntarle al grafo cuáles son sus dependencias, o al revés.

Diseño de la solución

Decisiones de diseño

Lo primero que tuve que decidir fue qué tipo de árbol usar. Al principio pensé en un ABB (árbol binario de búsqueda), pero me di cuenta que no encajaba porque la jerarquía de materias no es binaria: una facultad puede tener muchos departamentos, y un departamento muchas materias. Así que fui por un árbol general, donde cada nodo puede tener cualquier cantidad de hijos. Implementé la lista de hijos con una lista enlazada simple hecha a mano, sin usar ArrayList ni nada de Java, para cumplir con la exigencia de no depender de librerías.

Para el grafo, necesitaba modelar prerrequisitos. Como una materia puede tener varios prerrequisitos y además las dependencias son dirigidas (por ejemplo, "Estructura de Datos II" requiere "Estructura de Datos I", pero no al revés), elegí un grafo dirigido. Para evitar ciclos (una materia no puede ser requisito de sí misma indirectamente), usé un DAG (grafo acíclico dirigido). La representación la hice con lista de adyacencia porque es más eficiente en memoria cuando hay pocas aristas en comparación con el total de vértices, que es típico en un plan de estudios (cada materia tiene pocos prerrequisitos). La lista de adyacencia la implementé también con enlaces manuales, clase `NodoArista`.

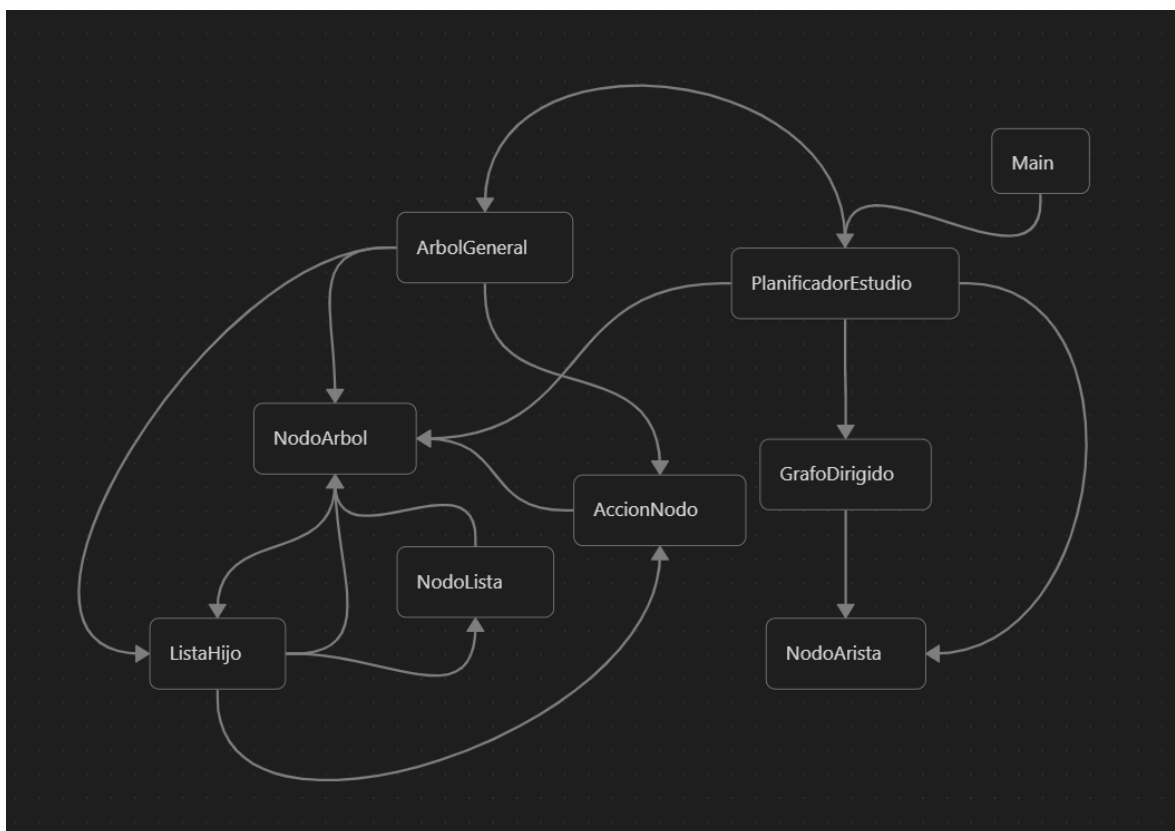
El algoritmo de grafo que implementé es el ordenamiento topológico usando DFS. La razón es que quería generar una secuencia de estudio que respete los prerrequisitos. Con el orden topológico, si empiezas desde las materias sin dependencias, puedes cursar en orden sin romper ninguna regla. También pensé en Dijkstra, pero no hacía falta porque los prerrequisitos no tienen pesos (o se podrían poner pesos como dificultad, pero lo dejé para después).

Relación entre el árbol y el grafo

La integración fue más sencilla de lo que pensaba. La clave está en que cada nodo del árbol tiene un ID (entero). Ese mismo ID se usa como identificador del vértice en el grafo. Entonces, cuando creo una materia nueva, la agrego al árbol (como nodo con su nombre e ID) y también agrego un vértice con ese ID en el grafo. Luego, para agregar un prerrequisito, solo uso los IDs: `grafo.agregarArista(idMateria, idPrerrequisito)`.

De esta forma, el árbol puede mostrar la jerarquía completa, y cuando el usuario consulta los prerrequisitos de una materia, el programa busca el ID en el grafo, obtiene la lista de IDs adyacentes, y después usa el árbol para traducir esos IDs a nombres legibles y mostrarlos. Al revés también funciona: el árbol puede listar todas las materias y para cada una preguntar al grafo si tiene dependencias.

Diagrama de clase



Implementación

Acá voy a mostrar las partes más importantes del código que hice. No voy a poner todo porque sería muy largo, pero sí lo principal para que se entienda cómo funciona cada estructura.

El árbol general

Primero, el nodo del árbol tiene un id, un nombre y una lista de hijos. La lista de hijos la implementé a mano con una clase ListaHijos que usa nodos enlazados simples. Esto fue medio tedioso pero me obligó a entender bien cómo funcionan las listas enlazadas.

```
public class NodoArbol { 36 usages WTHell
    private int id; 3 usages
    private String nombre; 3 usages
    private ListaHijos hijos; 5 usages
```

La lista de hijos tiene métodos como agregar, obtener, buscarPorNombre. Por ejemplo, el método agregar recorre hasta el final para insertar:

```
// Agregar un hijo al final de la lista
public void agregar(NodoArbol hijo) { 1 usage WTHell
    NodoLista nuevo = new NodoLista(hijo);
    if (cabeza == null) {
        cabeza = nuevo;
    } else {
        NodoLista actual = cabeza;
        while (actual.getSiguiende() != null) {
            actual = actual.getSiguiende();
        }
        actual.setSiguiende(nuevo);
    }
    size++;
}
```

Después la clase ArbolGeneral tiene la raíz y los métodos de inserción, búsqueda y recorridos. La búsqueda por id es recursiva:

```
private NodoArbol buscarPorIdRecursivo(NodoArbol actual, int id) { 2 usages WTHell
    if (actual == null) return null;
    if (actual.getId() == id) return actual;
    ListaHijos hijos = actual.getHijos();
    for (int i = 0; i < hijos.size(); i++) {
        NodoArbol encontrado = buscarPorIdRecursivo(hijos.obtener(i), id);
        if (encontrado != null) return encontrado;
    }
    return null;
}
```

Para recorrer el árbol usé preorden y postorden, y también un método imprimirArbol que muestra la jerarquía con indentación. La verdad que el método de imprimir me costó un poco porque hay que acordarse de aumentar el nivel al llamar a los hijos.

El grafo dirigido

Para el grafo, como quería representar dependencias, hice un grafo dirigido con lista de adyacencia usando la clase NodoArista.

```
public class NodoArista { 12 usages WTHell
    private int destino; 3 usages
    private NodoArista siguiente; 3 usages
}
```

El grafo tiene un arreglo de NodoArista de tamaño fijo (500). Cada posición del arreglo es la lista de adyacentes de ese vértice. Para agregar una arista, simplemente inserto al inicio de la lista:

```
// Agrega una arista dirigida origen -> destino
public void agregarArista(int origen, int destino) { 1 usage WTHell
    if (origen < 0 || origen >= MAX_VERTICES || destino < 0 || destino >= MAX_VERTICES) {
        System.out.println("Error: ID fuera de rango");
        return;
    }
    if (!existe[origen]) agregarVertice(origen);
    if (!existe[destino]) agregarVertice(destino);

    NodoArista nuevo = new NodoArista(destino);
    nuevo.setSiguiente(adyacencias[origen]);
    adyacencias[origen] = nuevo;
}
```

El algoritmo más importante es el ordenamiento topológico usando DFS. Lleva un arreglo de estados: 0 = no visitado, 1 = en pila (para detectar ciclos), 2 = procesado. Me costó entender al principio pero después salió.

```
private boolean dfsTopologico(int v, int[] estado, int[] resultado, int[] indice) { 2 usages WTHell *
    estado[v] = 1;
    NodoArista actual = adyacencias[v];
    while (actual != null) {
        int vecino = actual.getDestino();
        if (estado[vecino] == 1) return false;
        if (estado[vecino] == 0) {
            if (!dfsTopologico(vecino, estado, resultado, indice)) return false;
        }
        actual = actual.getSiguiete();
    }
    estado[v] = 2;
    resultado[indice[0]] = v;
    indice[0]--;
    return true;
}
```

Integración de árbol y grafo

La clase PlanificadorEstudios junta ambas estructuras. Al agregar una materia, la meto en el árbol y también en el grafo como vértice. Al agregar un prerrequisito, solo uso el grafo con los IDs. Para mostrar el orden topológico, obtengo el arreglo de IDs del grafo y después los convierto a nombres usando el árbol. Eso me pareció elegante porque no tuve que duplicar información.

```
// Muestra el orden topológico de estudio
public void mostrarOrdenTopologico() { 1 usage WTHell
    int[] orden = grafo.ordenTopologico();
    if (orden == null) {
        System.out.println("No se puede generar orden: hay ciclos en los prerrequisitos.");
        return;
    }
    System.out.println("Orden de estudio recomendado:");
    for (int i = 0; i < orden.length; i++) {
        NodoArbol nodo = arbol.buscarPorId(orden[i]);
        String nombre = (nodo != null) ? nodo.getNombre() : "Desconocido";
        System.out.println((i+1) + ". " + nombre + " (ID: " + orden[i] + ")");
    }
}
```

Análisis de complejidad

Acá voy a mirar las operaciones principales que hice y ver cuánto tardan y cuánta memoria usan. Tampoco voy a ponerme muy estricto, pero algo hay que mostrar.

Complejidad temporal

Árbol general:

Inserción: si inserto un hijo directamente bajo un nodo que ya tengo, es $O(1)$ porque solo agrego al final de la lista de hijos. Pero si tengo que buscar el padre por id o nombre, primero hay que buscarlo, y esa búsqueda puede ser $O(n)$ en el peor caso, donde n es la cantidad total de nodos. Así que la inserción completa es $O(n)$ por la búsqueda.

Búsqueda por id o nombre: tengo que recorrer recursivamente todo el árbol en el peor caso, así que es $O(n)$. No tengo ordenamiento porque es árbol general, no ABB.

Recorrido preorden o postorden: visito cada nodo exactamente una vez, así que es $O(n)$. Eso está bien.

Grafo dirigido:

Agregar vértice: solo marco un booleano en un arreglo, es $O(1)$.

Agregar arista: inserto al inicio de la lista de adyacencia, también $O(1)$. Pero antes valido que los IDs estén dentro del rango, eso es $O(1)$.

Ordenamiento topológico con DFS: recorro todos los vértices y todas las aristas. Si V es el número de vértices y E el de aristas, la complejidad es $O(V + E)$. En mi caso, como el grafo es un DAG, no hay ciclos y funciona bien. El peor caso sería cuando hay que explorar todo.

Mostrar adyacentes: recorro la lista de un vértice. En el peor caso podría ser $O(V)$ si un vértice tiene muchas aristas, pero normalmente son pocas.

Integración (Planificador):

Mostrar orden topológico: el grafo da el arreglo de IDs en $O(V+E)$, y luego por cada ID busco en el árbol para obtener el nombre. Esa búsqueda en el árbol es $O(n)$ cada una, pero n es la cantidad de materias, que sería lo mismo que V (porque cada materia es un vértice). Así que queda $O(V * V) = O(V^2)$ si lo hago así. La verdad es que eso es medio ineficiente. Podría haber hecho un mapa interno pero no usamos HashMap porque es de Java. Para mejorarlo, podría haber guardado los nodos en un arreglo auxiliar, pero no lo hice. En mi defensa, el plan de estudios no tiene miles de materias, así que no importa tanto.

Complejidad espacial

Árbol: Cada nodo guarda un id (int), un nombre (String) y una lista de hijos. La lista de hijos usa nodos enlazados, así que por cada hijo tengo un NodoLista adicional. En total, para n nodos, uso $O(n)$ espacio más lo que ocupe cada String. Los Strings dependen del nombre, pero no los cuento tanto.

Grafo: Tengo un arreglo de tamaño fijo (500) de listas de adyacencia. Cada arista agrega un NodoArista. Si hay V vértices y E aristas, el espacio es $O(V + E)$. Como puse un máximo de 500, en la práctica el espacio está acotado.

Total: $O(n + V + E)$ que está bien para un proyecto así.

Datos de prueba

```
private static void cargarDatosEjemplo(PlanificadorEstudios p) { 1 usage 2 WTHell
    // Árbol: Facultad -> Departamento -> Materias
    p.agregarMateria(id: 1, nombre: "Informática", padreId: null); // raíz
    p.agregarMateria(id: 2, nombre: "Programación", padreId: 1);
    p.agregarMateria(id: 3, nombre: "Matemáticas", padreId: 1);
    p.agregarMateria(id: 4, nombre: "Programación I", padreId: 2);
    p.agregarMateria(id: 5, nombre: "Estructura de Datos I", padreId: 2);
    p.agregarMateria(id: 6, nombre: "Estructura de Datos II", padreId: 2);
    p.agregarMateria(id: 7, nombre: "Álgebra", padreId: 3);
    p.agregarMateria(id: 8, nombre: "Cálculo", padreId: 3);

    // Grafo de prerrequisitos
    p.agregarPrerrequisito(idMateria: 5, idPrerrequisito: 4); // ED I requiere Prog I
    p.agregarPrerrequisito(idMateria: 6, idPrerrequisito: 5); // ED II requiere ED I
    p.agregarPrerrequisito(idMateria: 6, idPrerrequisito: 7); // ED II requiere Álgebra (por decir)
    p.agregarPrerrequisito(idMateria: 8, idPrerrequisito: 7); // Cálculo requiere Álgebra
}
```

===== PLANIFICADOR DE ESTUDIOS =====

1. Mostrar árbol jerárquico de materias
2. Agregar nueva materia
3. Agregar prerrequisito
4. Ver prerrequisitos de una materia
5. Mostrar orden topológico de estudio
6. Salir

Opción: 1

- 1 - Informática
 - 2 - Programación
 - 4 - Programación I
 - 5 - Estructura de Datos I
 - 6 - Estructura de Datos II
 - 3 - Matemáticas
 - 7 - Álgebra
 - 8 - Cálculo

===== PLANIFICADOR DE ESTUDIOS =====

1. Mostrar árbol jerárquico de materias
2. Agregar nueva materia
3. Agregar prerrequisito
4. Ver prerrequisitos de una materia
5. Mostrar orden topológico de estudio
6. Salir

Opción: 4

ID de la materia: 8

Prerrequisitos de 8:

- Álgebra (ID: 7)

```
===== PLANIFICADOR DE ESTUDIOS =====  
1. Mostrar árbol jerárquico de materias  
2. Agregar nueva materia  
3. Agregar prerrequisito  
4. Ver prerrequisitos de una materia  
5. Mostrar orden topológico de estudio  
6. Salir  
Opción: 3  
ID de la materia que requiere prerrequisito: 8  
ID del prerrequisito: 6
```

```
===== PLANIFICADOR DE ESTUDIOS =====  
1. Mostrar árbol jerárquico de materias  
2. Agregar nueva materia  
3. Agregar prerrequisito  
4. Ver prerrequisitos de una materia  
5. Mostrar orden topológico de estudio  
6. Salir  
Opción: 4  
ID de la materia: 8  
Prerrequisitos de 8:  
- Estructura de Datos II (ID: 6)  
- Álgebra (ID: 7)
```

Conclusiones

Terminar este proyecto me dejó varias cosas. Primero, aprendí bien la diferencia entre un árbol general y un ABB, y por qué uno u otro se usa según el problema. Al principio quería hacer un ABB porque sonaba más técnico, pero después me di cuenta que la jerarquía de materias no es binaria y forzarlo hubiera sido un lío. El árbol general con lista de hijos casera fue más natural, aunque implementar la lista enlazada a mano me hizo acordarme de los punteros y esas cosas que veíamos en estructura de datos I.

Con el grafo también tuve momentos de frustración, sobre todo con el ordenamiento topológico recursivo. Se me olvidó marcar bien los estados y por un tiempo no detectaba ciclos, pero después de depurar con hartos `System.out.println` logré que funcionara. Eso sí, cuando detecta un ciclo ya sé que es porque alguien puso mal un prerrequisito.

Lo que más me gustó fue la integración. Usar el mismo ID en el árbol y el grafo hizo que todo encajara sin tener que andar duplicando información. Pude mostrar el orden topológico con nombres bonitos consultando el árbol, y también al revés, mostrar los prerrequisitos de una materia directamente. Creo que esa parte quedó sólida.

Dificultades tuve varias: al principio el método de imprimir el árbol no era público y no entendía por qué no compilaba; después me di cuenta que faltaba el método público que llamara al privado. También me equivoqué con los índices al recorrer la lista de hijos, pero nada grave. Otra cosa que podría mejorar es el rendimiento al mostrar el orden topológico, porque por cada ID vuelvo a buscar en el árbol y eso es medio lento si hay muchas materias. Podría hacer un arreglo auxiliar que guarde los nodos por ID, pero como no usamos `HashMap` me dio pereza.

En general, el proyecto cumple con todo lo que pedía la guía: árbol general con inserción, búsqueda y recorridos; grafo dirigido con lista de adyacencia y ordenamiento topológico; y ambos colaborando en la clase `PlanificadorEstudios`.

Referencias bibliográficas

- Bailey, D. (s.f.). Java Structures: Data Structures in Java for the Principled Programmer (3ra ed.). Department of Computer Science, Williams College. Disponible en: <https://www.cs.williams.edu/~bailey/JavaStructures/>
- Morin, P. (2011). Open Data Structures (in Java) (ed. 0.1G). Disponible en: <http://opendatastructures.org/>
- LeetCode. (s.f.). Course Schedule II – Topological Sort (Kahn's Algorithm). Disponible en: <https://leetcode.com/problems/course-schedule-ii/>
- Miller, B. N., & Ranum, D. L. (s.f.). Problem Solving with Algorithms and Data Structures using Java. Runestone Academy. Disponible en: <https://runestone.academy/>
- GitHub. Repositorios consultados: Sloot25/EstructurasDeDatos y davidcv-dev/EstructurasDeDatos. Disponible en:
<https://github.com/Sloot25/EstructurasDeDatos> y
<https://github.com/davidcv-dev/EstructurasDeDatos>
- makigas.es. (2015). Listas enlazadas en Java – Estructuras de datos. Disponible en: <https://www.makigas.es/>
- Wikipedia. (s.f.). Grafo acíclico dirigido. Disponible en: https://es.wikipedia.org/wiki/Grafo_ac%C3%ADclico_dirigido
- alg0.dev. (s.f.). Ordenamiento Topológico (Algoritmo de Kahn). Disponible en: <https://www.alg0.dev/es/topological-sort>